Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression**	
Experiment No: 02	**Date:**	**Enrollment No: 92410133012**

Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression

IDE: Google Colab

Theory:

Linear regression is a method for determining the best linear relationship between two variables *X* and *Y*. If variables *X* and *Y* are uncorrelated, it is pointless embarking upon linear regression. However, if a reasonable degree of correlation exists between *X* and *Y* then linear regression may be a useful means to describe the relationship between the two variables. The usual approach is to use the *least-squares* method, which minimizes the squared difference between the actual data points and a straight line. Let $[x_i, y_i]$, $i = 1,2,3,\ldots,N$ be the *N* pairs of data values of the variables *X* and *Y*. The straight-line relating *X* and *Y* is $y = mx + c$, where *m* and *c* are the gradient and constant values (to be determined) defining the straight line. Thus, $y(x_i) - y_i$ is the difference between the line and data point *i* (see Fig. 1). Taking all the data points, we seek values of *m* and *c* that minimize the squared difference *SD*.

$$\sum_{1}^{N} \left[y(x_i) - y_i \right]^2$$

This is achieved by calculating the partial derivatives of *SD* with respect to *m* and *c* and finding the pair [*m*,*c*] such that *SD* is at a minimum.

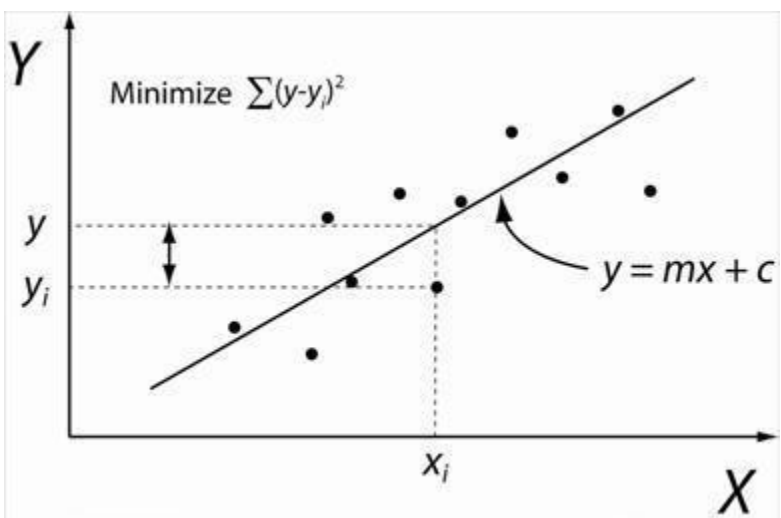


Figure 1: Illustration of Linear Regression. Linear least squares regression, the idea is to find the line y = mx + c that minimizes the mean squared difference between the line and the data points

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression**	
Experiment No: 02	**Date:**	**Enrollment No: 92410133012**

Batch Gradient Descent:

Gradient Descent is an optimization algorithm used for minimizing the cost function in various machine learning algorithms. It is basically used for updating the parameters of the learning model. Batch gradient descent which processes all the training examples for each iteration of gradient descent. But if the number of training examples is large, then batch gradient descent is computationally very expensive.

Let m be the number of training examples. Let n be the number of features.

Algorithm for batch gradient descent :

Let $h_\theta(x)$ be the hypothesis for linear regression. Then, the cost function is given by:
Let Σ represents the sum of all training examples from i=1 to m.
$J_{train}(\theta) = (1/2m)\ \Sigma(\ h_\theta(x^{(i)})\ -\ y^{(i)})^2$

Repeat {
$\theta j = \theta j - (\text{learning rate}/m) * \Sigma(\ h_\theta(x^{(i)})\ -\ y^{(i)})x_j^{(i)}$
For every j =0 …n
}
Where $x_j^{(i)}$ Represents the jth feature of the ith training example. So if *m* is very large(e.g. 5 million training samples), then it takes hours or even days to converge to the global minimum.That's why for large datasets, it is not recommended to use batch gradient descent as it slows down the learning.

Pre Lab Exercise:

a. Explain the meaning of linear regression

Linear Regression is a **supervised machine learning algorithm** that is used to find the relationship between **independent variables (features)** and a **dependent variable (output)**.

It works by fitting a **straight line (best fit line)** through the given data points so that the difference between the predicted values and the actual values is minimized.

The equation of a simple linear regression line is:

$$y = mx + c$$

where

- **x** = input feature
- **y** = predicted output
- **m** = slope of the line
- **c** = intercept

The goal of linear regression is to **predict the output value based on the input feature**

b. Write three applications of linear regression

- **House Price Prediction**

Linear regression can predict house prices based on factors like size, location, and number of rooms.

- **Sales Forecasting**

Companies use linear regression to predict future sales based on past sales data.

- **Stock Market Analysis**

It can be used to analyze trends and predict future stock prices based on historical data.

c. Write three advantages of linear regression

1. **Simple to understand and implement**
   Linear regression is one of the easiest machine learning algorithms to use.
2. **Fast training process**
   It requires less computational power compared to many other algorithms.
3. **Easy interpretation**
   The relationship between variables can be easily understood using the regression line.

d. Write three limitations of linear regression

1. **Assumes linear relationship**
   It only works well when the relationship between variables is linear.
2. **Sensitive to outliers**
   Extreme values can affect the regression line and reduce accuracy.
3. **Not suitable for complex datasets**
   It may not perform well when the data has complex patterns.

Methodology:

1. Load the basic libraries and packages
2. Load the dataset
3. Analyse the dataset
4. Pre-process the data
5. Visualize the Data
6. Separate the feature and prediction value columns
7. Write the Hypothesis Function
8. Write the Cost Function
9. Write the Gradient Descent optimization algorithm
10. Apply the training over the dataset to minimize the loss
11. Find the best fit line to the given dataset
12. Observe the cost function vs iterations learning curve

Program (Code):

To be attached with

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression**	
Experiment No: 02	**Date:**	**Enrollment No: 92410133012**

```
# Step 1: Import basic libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Step 2: Load the dataset
try:
    data = pd.read_csv("data.csv")
except FileNotFoundError:
    print("Error: 'data.csv' not found. Please upload the file or provide the correct path.")
    print("Generating a dummy dataset for demonstration purposes.")
    np.random.seed(0) # for reproducibility
    X_dummy = np.random.rand(100, 1) * 10
    y_dummy = 2 * X_dummy + 1 + np.random.randn(100, 1) * 2
    data = pd.DataFrame({'x': X_dummy.flatten(), 'y': y_dummy.flatten()})

# Step 3: Analyse the dataset
print("First 5 rows of dataset:")
print(data.head())

print("\nDataset shape:")
print(data.shape)

print("\nStatistical summary:")
print(data.describe())

# Step 4: Pre-process the data
data = data.dropna()

# Step 5: Visualize the data
plt.scatter(data['x'], data['y'])
plt.title("Scatter Plot of Data Points D12")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()

# Step 6: Separate feature and target columns
X = data['x'].values
Y = data['y'].values

m = len(X)
```

```
# Step 7: Hypothesis function
def hypothesis(x, theta0, theta1):
    return theta0 + theta1 * x

# Step 8: Cost function
def compute_cost(X, Y, theta0, theta1):
    total_error = 0
    for i in range(m):
        total_error += (hypothesis(X[i], theta0, theta1) - Y[i])**2
    return total_error / (2*m)

# Step 9: Gradient Descent Algorithm
def gradient_descent(X, Y, theta0, theta1, learning_rate, iterations):

    cost_history = []

    for i in range(iterations):
        sum_error0 = 0
        sum_error1 = 0

        for j in range(m):
            error = hypothesis(X[j], theta0, theta1) - Y[j]
            sum_error0 += error
            sum_error1 += error * X[j]

        theta0 = theta0 - (learning_rate/m) * sum_error0
        theta1 = theta1 - (learning_rate/m) * sum_error1

        cost = compute_cost(X, Y, theta0, theta1)
        cost_history.append(cost)

    return theta0, theta1, cost_history

# Step 10: Apply training
theta0 = 0
theta1 = 0
learning_rate = 0.01
iterations = 100

theta0, theta1, cost_history = gradient_descent(X, Y, theta0, theta1, learning_rate, iterations)

print("Theta0:", theta0)
print("Theta1:", theta1)
```

```
# Step 11: Plot best fit line
predictions = theta0 + theta1 * X

plt.scatter(X, Y, color='blue')
plt.plot(X, predictions, color='green')
plt.title("Linear Regression Best Fit Line")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()

# Step 12: Cost function vs iterations
plt.plot(range(iterations), cost_history)
plt.title("Cost Function vs Iterations")
plt.xlabel("Iterations")
plt.ylabel("Cost")
plt.show()
```

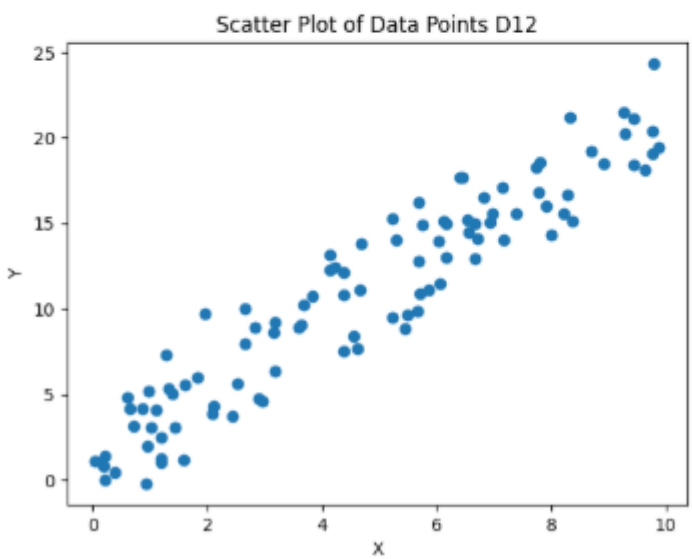
Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression**	
Experiment No: 02	**Date:**	**Enrollment No: 92410133012**

Results:

To be attached with

a. Datapoints scattering (without best fit line)

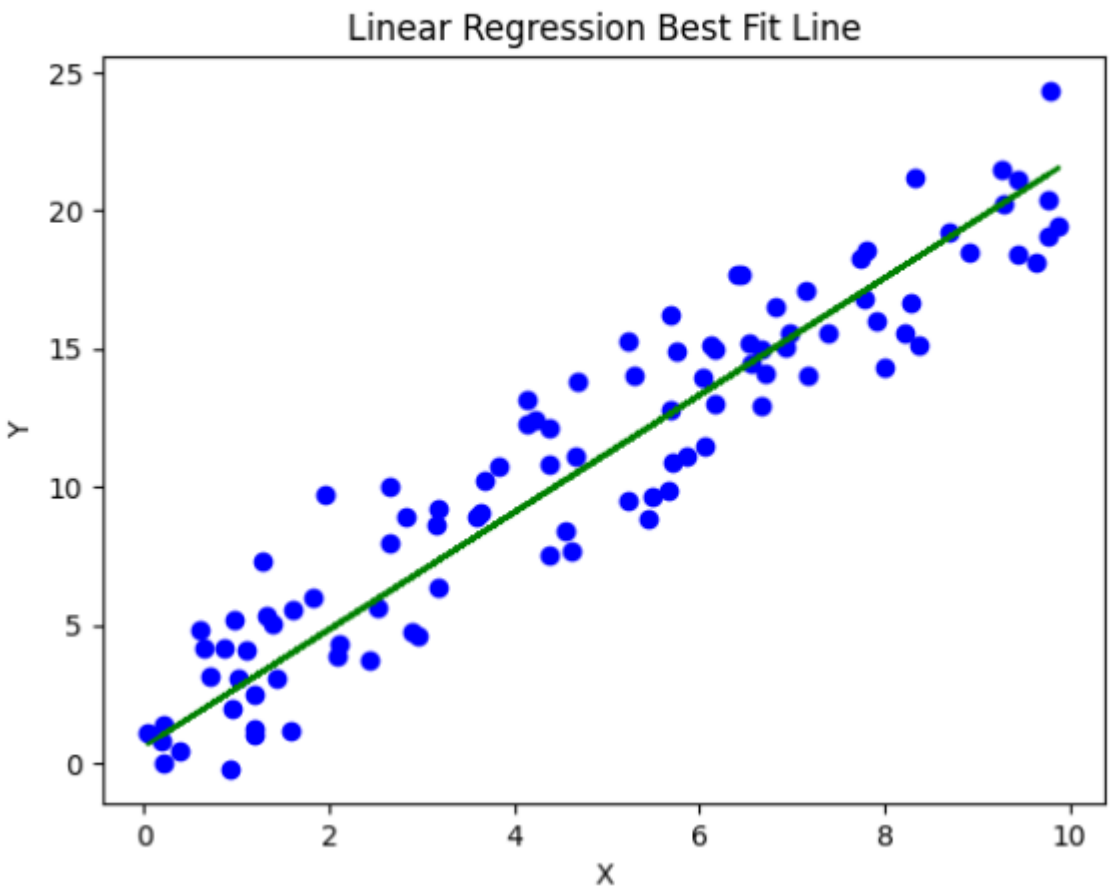
```
Generating a dummy dataset for demonstration purposes:
First 5 rows of dataset:
          x          y
0  5.488135   9.645970
1  7.151894  17.105440
2  6.027634  13.986592
3  5.448832   8.825176
4  4.236548  12.449600

Dataset shape:
(100, 2)

Statistical summary:
                x           y
count  100.000000  100.000000
mean     4.727938   10.840545
std      2.897540    6.096767
min      0.046955   -0.210732
25%      2.058032    5.306228
50%      4.674810   10.991166
75%      6.844833   15.349413
max      9.883738   24.338668
```



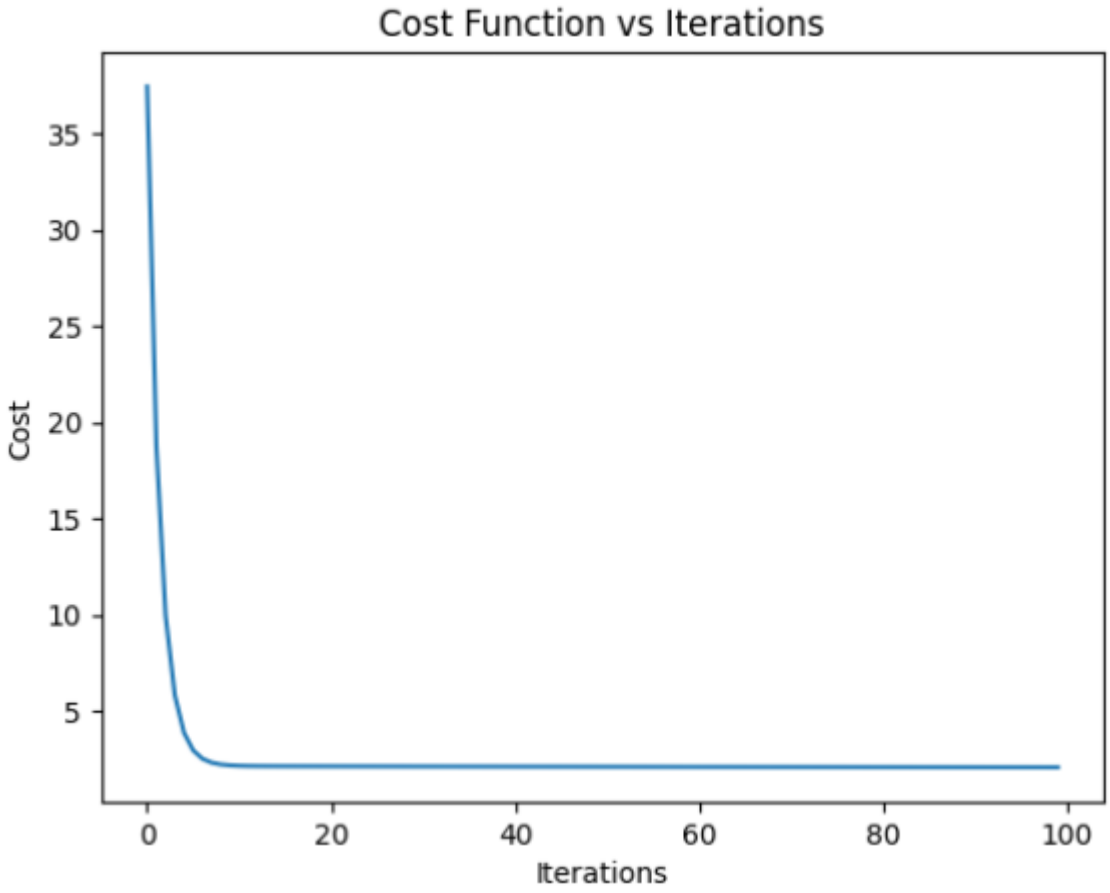

```
Theta0: 0.5939680520965535
Theta1: 2.1196325203963564
```

b. Scatter plot showing the Best fit line in the last iteration of training

```
Theta0: 0.5939680520965535
Theta1: 2.1196325203963564
```



Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression**	
Experiment No: 02	**Date:**	**Enrollment No: 92410133012**



Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression**	
Experiment No: 02	**Date:**	**Enrollment No: 92410133012**

Observation and Result Analysis:

a. Nature of the dataset

The dataset contains **scattered data points representing the relationship between the input feature (X) and the output value (Y)**.

From the scatter plot, we can observe that the data points follow a **roughly increasing trend**, which indicates a **positive linear relationship** between the variables.

This makes the dataset suitable for applying the **Linear Regression algorithm** to find the best fit line.

b. During Training Process

During the training process, the **gradient descent algorithm** repeatedly updates the parameters (θ values) to minimize the cost function.

At each iteration:

- The predicted values are calculated using the hypothesis function.
- The error between predicted values and actual values is measured.
- The parameters are updated to reduce this error.

As the iterations increase, the **cost function gradually decreases**, showing that the model is learning and improving.

c. After the training Process

After training is completed, the algorithm finds the **best fit line** that represents the relationship between the input feature and the output value.

This line minimizes the prediction error and passes through the region where most data points are located.

The trained model can now be used to **predict output values for new input data**.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression**	
Experiment No: 02	**Date:**	**Enrollment No: 92410133012**

Post Lab Exercise:

a. What are the major assumptions considered in linear regression

The main assumptions of linear regression are:

1. **Linearity** – There should be a linear relationship between input and output variables.
2. **Independence** – The observations should be independent of each other.
3. **Constant Variance (Homoscedasticity)** – The variance of errors should remain constant.
4. **Normal Distribution of Errors** – The error terms should follow a normal distribution.

b. Why MSE is used instead of MAE for calculating the loss function

Mean Squared Error (MSE) is commonly used because:

- It **penalizes larger errors more strongly** by squaring them.
- It is **differentiable**, which makes gradient descent optimization easier.
- It provides smoother mathematical calculations during training.

Because of these reasons, MSE is often preferred over MAE in linear regression.

c. How can the behaviour of outliers be understood while dealing with the unseen dataset

Outliers can be detected by:

- **Using box plots**
- **Using statistical methods like IQR**
- **Analyzing scatter plots**

If extreme values are observed far from the majority of data points, they are considered outliers. These outliers can significantly affect the regression line and should be handled carefully during data preprocessing.

d. Derive the Normal Equation for the Linear Regression.

The **Normal Equation** is used to directly calculate the optimal parameters of linear regression without using gradient descent.

The hypothesis function is:

$$h_\theta(x) = X\theta$$

The cost function is:

$$J(\theta) = \frac{1}{2m}(X\theta - y)^T(X\theta - y)$$

To minimize the cost function, we take the derivative with respect to θ and set it equal to zero. After solving, we obtain the Normal Equation:

$$\theta = (X^T X)^{-1} X^T y$$

Where:

- **X** = feature matrix
- **y** = output vector
- $\mathbf{X^T}$ = transpose of X
- $\mathbf{(X^TX)^{-1}}$ = inverse of the matrix

This equation directly gives the **best parameters for the regression model**.